



Universidad del Valle de Guatemala
Facultad de Ingeniería
CC3092 — Deep Learning y Sistemas Inteligentes

Laboratorio #6

Transfer Learning y Fine-Tuning

Clasificación de CIFAR-10 con una CNN desde cero y VGG-16 preentrenada

Nicolás Concuá - 23197
Esteban Cárcamo - 23016

Guatemala, septiembre de 2026

Enlace al repositorio: <https://github.com/nicoCT04/Lab6-DL>

1. Datos y preparación

CIFAR-10 tiene 60,000 imágenes RGB de 32×32 (50,000 de entrenamiento y 10,000 de test) en 10 clases perfectamente balanceadas (5,000 y 1,000 por clase). La media por canal del entrenamiento (**0.491, 0.482, 0.447**) y su desviación (**0.247, 0.244, 0.262**) son cercanas a las de ImageNet (0.485, 0.456, 0.406 / 0.229, 0.224, 0.225), pues ambos son fotografías naturales. Se dividió el entrenamiento en 45,000/5,000 (estratificado) y el test oficial se usó una sola vez.

- **Pipelines.** CNN desde cero: resolución nativa 32×32 + Normalize(CIFAR). VGG-16: **Resize a 112×112** + Normalize(ImageNet), sin el CenterCrop de **weights.transforms()** (recortaría el objeto). Pasar de 32×32 (1,024 px) a 224×224 (50,176 px) multiplica píxeles y cómputo por 49; a 112×112 el factor es 12.25 y el costo es 4× menor que a 224. Se eligió 112 para ambos VGG por usar una GPU integrada (Apple M4 Pro, backend MPS).
- **Data augmentation.** RandomCrop(32, padding=4, reflect) y RandomHorizontalFlip: traslaciones pequeñas y espejo no cambian la clase y reducen el overfitting. Solo se aplica a entrenamiento: validación y test deben medir el desempeño sobre la distribución real, de forma determinista. Para no saturar la CPU, las transformaciones se implementaron por batch en la GPU (verificadas contra torchvision).
- **Pares difíciles esperados.** gato–perro, automóvil–camión, venado–caballo y avión–barco: comparten forma, textura o fondo, y a 32×32 se pierden los detalles que los distinguen.

2. Investigación: transfer learning en PyTorch

- **vgg16 y VGG16_Weights.** **vgg16(weights=VGG16_Weights.IMAGENET1K_V1)** construye VGG-16 (13 conv 3×3 + 3 FC, 138 M parámetros) con pesos de ImageNet; **weights=None** la inicializa al azar. **weights.transforms()** devuelve el preprocesamiento usado al entrenar: Resize(256) → CenterCrop(224) → [0,1] → Normalize(ImageNet) [1][2].
- **Estructura.** **features** contiene los 5 bloques conv (cada uno cierra con MaxPool); **avgpool** es un AdaptiveAvgPool2d((7,7)) que fija la salida en 7×7 para cualquier resolución (a 112 px el mapa es 3×3; en MPS se sustituyó por un módulo matricial equivalente); **classifier** son 3 capas lineales 25088→4096→4096→1000 con ReLU y Dropout; la última se reemplaza por Linear(4096, 10).
- **requires_grad.** Si es False, autograd no calcula el gradiente del parámetro y el optimizador no lo actualiza. Se congela un bloque con **features[i:j].requires_grad_(False)**; si todo lo inferior está congelado, no se hace backward por esa parte [3].
- **train() / eval() / no_grad().** **train()** y **eval()** cambian el modo de Dropout y BatchNorm (no congelan pesos); **torch.no_grad()** desactiva el grafo de autograd para validar e inferir con menos memoria y tiempo [3].
- **param_groups.** El optimizador acepta una lista de grupos con su propio lr, p. ej. **SGD([{'params': backbone, 'lr': 1e-3}, {'params': head, 'lr': 1e-2}])**: lr pequeño para no destruir las features preentrenadas y lr mayor para la cabeza nueva [4].
- **Conteo de parámetros y operaciones.** **torchinfo.summary** lista forma de salida, parámetros (totales/entrenables) y mult-adds por capa; **fvcore FlopCountAnalysis** traza el modelo y cuenta MACs por módulo (fvcore llama "flop" a un MAC) [5].
- **Memoria y tiempo en GPU.** Los kernels de CUDA son asíncronos: se llama **torch.cuda.synchronize()** antes de medir tiempo; **torch.cuda.max_memory_allocated()** da el pico de memoria de tensores. En Apple Silicon se usaron **torch.mps.synchronize()** y **torch.mps.current_allocated_memory()** (muestreada en cada paso) [6].
- **MACs vs. FLOPs; parámetros totales vs. entrenables.** Un MAC es una multiplicación-acumulación ($a \leftarrow a + b \cdot c$), así que 1 MAC $\approx$ 2 FLOPs; el backward cuesta $\approx 2\times$ el forward, por lo que entrenar una capa cuesta $\approx 3\times$ su forward. Los parámetros totales determinan memoria y costo de inferencia; los entrenables (**requires_grad=True**) determinan gradientes, estado del optimizador y cuántos datos se necesitan para no sobreajustar.

3. Modelos, iteraciones y resultados en validación

Se entrenaron tres modelos con 10 iteraciones en total (Tabla 1). **CNN desde cero:** 3 bloques conv (Conv3×3–BN–ReLU, MaxPool y Dropout) + 2 capas densas, a 32×32. **VGG-16 feature extractor:** los 5 bloques conv congelados; solo se entrena el clasificador (fc6–fc8, con fc8 → 10 salidas). **VGG-16 fine-tuning:** se descongelan los bloques superiores con SGD y dos grupos de parámetros (lr 1e-2 en el clasificador y 1e-3 o 1e-4 en el backbone). Todas las métricas son macro sobre validación (5,000 imágenes) y se conserva el checkpoint del mejor epoch según F1. La memoria es el pico de tensores en la GPU (MPS).

Tabla 1. Iteraciones (métricas macro en validación en el mejor epoch; en negrita la mejor de cada modelo)

Iter.	Configuración	Aug	Ep (mejor)	Acc	Prec	Rec	F1	Params / entren. (M)	s/epoch	Total (min)	Mem (GB)
CNN-1	32-64-128 ×1, adam 0.001	no	15 (11)	0.763	0.763	0.763	0.762	0.6 / 0.6	2.2	0.5	0.22
CNN-2	32-64-128 ×1, BN, drop 0.1/0.3, adamw 0.001 cosine	sí	25 (20)	0.822	0.821	0.822	0.820	0.6 / 0.6	2.5	1.0	0.26
CNN-3	64-128-256 ×2, BN, drop 0.2/0.4, sgd 0.1 onecycle	sí	30 (30)	0.925	0.925	0.925	0.925	2.2 / 2.2	11.6	5.8	0.45
FE-1	congeladas: b1–b5; lr 0.01	no	5 (5)	0.879	0.880	0.879	0.879	134.3 / 119.6	83.8	7.0	1.61
FE-2	congeladas: b1–b5; lr 0.01	sí	5 (4)	0.872	0.872	0.872	0.872	134.3 / 119.6	85.1	7.1	1.61
FE-3	congeladas: b1–b5 + fc6–7; lr 0.01	sí	5 (5)	0.828	0.829	0.828	0.827	134.3 / 0.04	73.3	6.1	0.70
FT-1	congeladas: b1–b4; lr bb 0.001 / cls 0.01	sí	5 (5)	0.902	0.902	0.902	0.901	134.3 / 126.7	103.5	8.6	1.71
FT-2	congeladas: b1–b3; lr bb 0.001 / cls 0.01	sí	5 (5)	0.926	0.926	0.926	0.926	134.3 / 132.6	153.6	12.8	1.92
FT-3	congeladas: b1–b3; lr bb 0.0001 / cls 0.01	sí	5 (5)	0.902	0.903	0.902	0.902	134.3 / 132.6	155.8	13.0	1.92
FT-4	congeladas: b1–b2; lr bb 0.001 / cls 0.01	sí	5 (5)	0.934	0.934	0.934	0.934	134.3 / 134.0	210.5	17.5	2.27

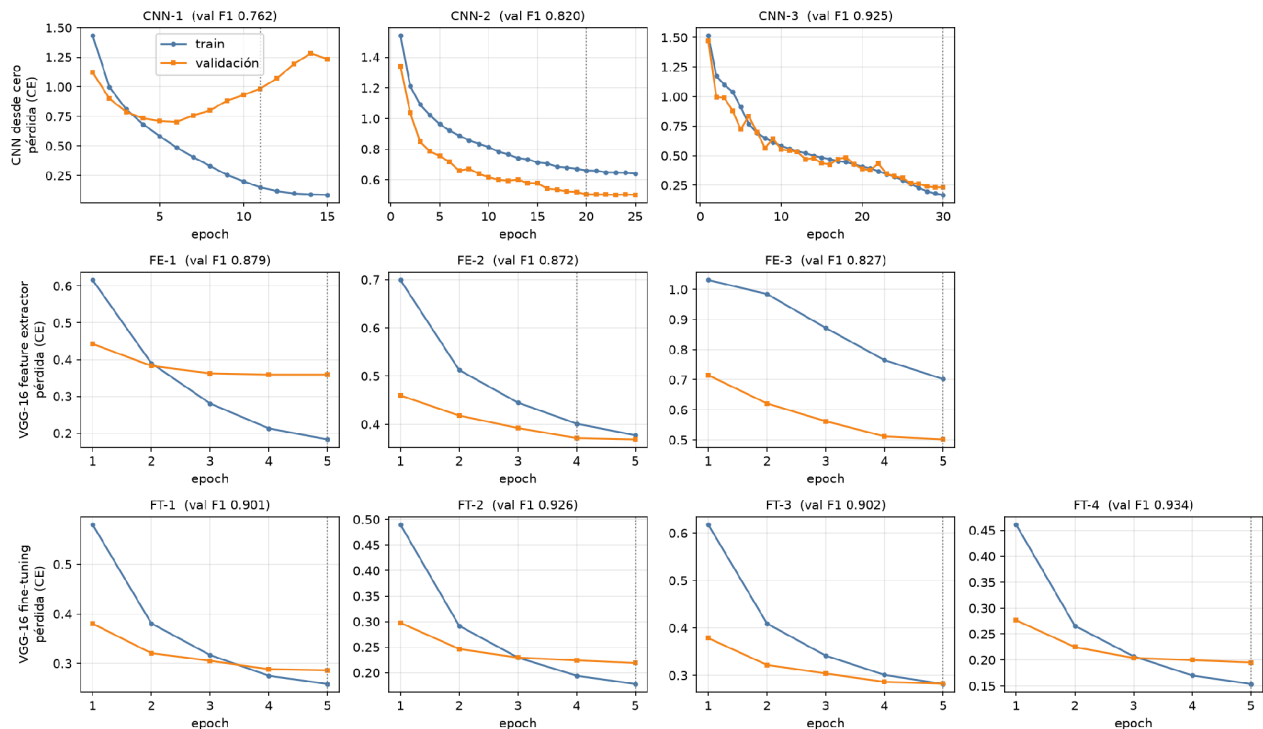


Figura 1. Pérdida de entrenamiento y validación por iteración (línea punteada: mejor epoch).

Curvas. CNN-1 (sin regularización) muestra overfitting claro: su pérdida de validación sube de 0.70 a 1.28 mientras la de entrenamiento llega a 0.09. BN, dropout y augmentation (CNN-2) lo eliminan, pero con 0.62 M parámetros hay underfitting; la red más ancha con OneCycle (CNN-3) sube a 0.925. En FE, la augmentation no ayudó en 5 epochs y el linear probe (FE-3) subajusta. En FT no hay catastrophic forgetting (la validación nunca cae), solo overfitting leve a partir del epoch 3.

4. Comparación de rendimiento y recursos

Se eligió la mejor iteración de cada modelo según el F1 de validación y se evaluó una única vez sobre el test oficial (10,000 imágenes). Los MACs se contaron con fvcare (1 MAC = 2 FLOPs) y la latencia se midió con sincronización de la GPU. El cómputo de entrenamiento por imagen es el forward completo (2·MACs) más un backward $\approx 2\times$ el forward **solo en la región por la que fluye el gradiente**: toda la red en la CNN, solo el clasificador en el FE y los bloques 3–5 + el clasificador en el FT. Se multiplicó por 45,000 imágenes y por los epochs entrenados. El experimento de datos reentrena cada configuración con el 10 % estratificado (4,500 imágenes, el doble de epochs).

Tabla 2. Comparación de los modelos finales (hardware: Apple M4 Pro (MPS))

Métrica	CNN desde cero (CNN-3)	VGG-16 feat. extractor (FE-1)	VGG-16 fine-tuning (FT-4)
Parámetros totales / entrenables (M)	2.20 / 2.20	134.30 / 119.59	134.30 / 134.04
Inferencia: GMACs / GFLOPs por imagen	0.15 / 0.31	3.96 / 7.91	3.96 / 7.91
Latencia GPU batch 1 / batch 256 (ms/img)	0.50 / 0.071	4.45 / 1.611	4.42 / 1.618
Tiempo por epoch (s)	11.6	83.8	210.5
Epochs hasta mejor val / entrenados	30 / 30	5 / 5	5 / 5
Tiempo total de entrenamiento (min)	5.8	7.0	17.5
Memoria pico GPU (GB)	0.45	1.61	2.27
Entrenamiento: GFLOPs por imagen	0.93	8.39	19.03
Cómputo total de entrenamiento (PFLOPs)	1.25	1.89	4.28
Test: accuracy	0.918	0.877	0.934
Test: precision / recall (macro)	0.918 / 0.918	0.877 / 0.877	0.934 / 0.934
Test: F1 macro	0.918	0.877	0.934
Test F1 con 10 % de datos (caída)	0.787 (−13.1 pts)	0.820 (−5.7 pts)	0.883 (−5.1 pts)

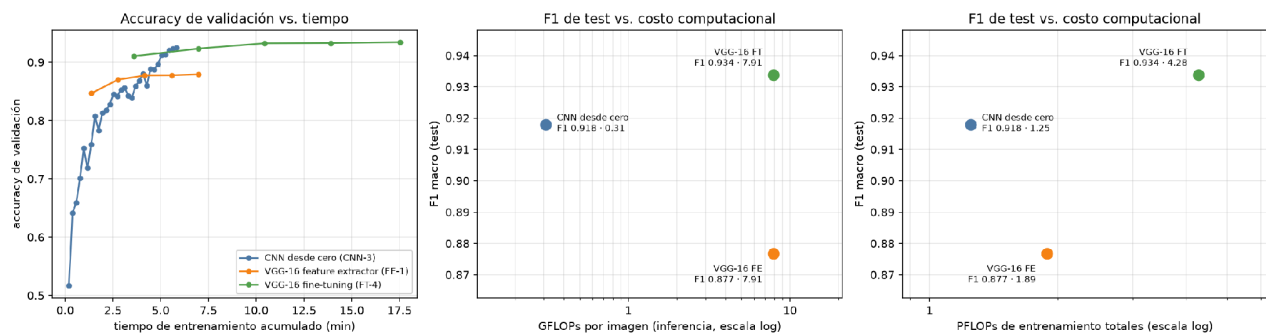


Figura 2. Accuracy de validación vs. tiempo acumulado y F1 de test vs. FLOPs de inferencia y de entrenamiento.

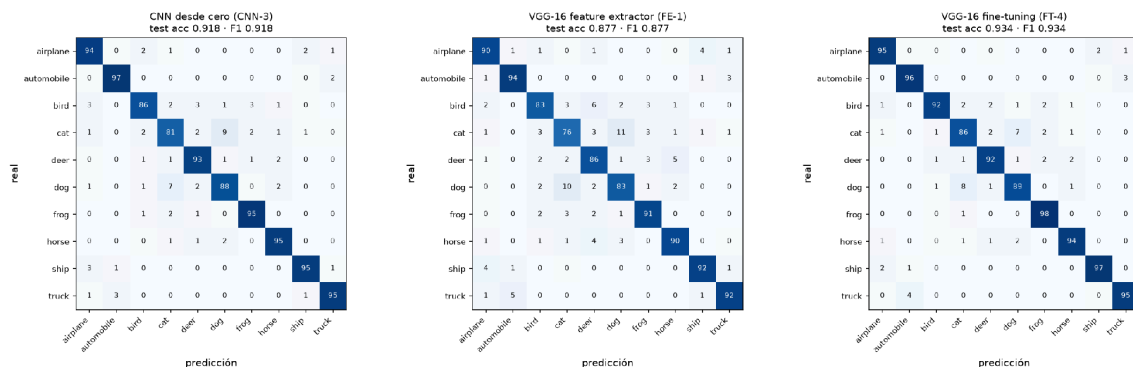


Figura 3. Matrices de confusión normalizadas sobre test (% por clase real).

5. Discusión y análisis

- **Mejor modelo y su costo.** VGG-16 con fine-tuning (FT-4) logró el mejor test (accuracy y F1 0.934), pero solo +1.6 puntos sobre la CNN desde cero (0.918) a cambio de 61× más parámetros, 26× más FLOPs de inferencia, $\sim 9\times$ más latencia (batch 1), 3.4× más cómputo y 3× más tiempo de entrenamiento y 5× más memoria. Con los datos completos, la mejora rara vez justifica ese costo.

- **Mejor relación desempeño/recursos: la CNN desde cero.** En la Figura 2 queda arriba a la izquierda: 98 % del F1 del mejor modelo con ~4 % de sus FLOPs. En accuracy vs. tiempo, el FT arranca en 0.91 (primer epoch, 3.5 min) pero la CNN lo alcanza hacia los 5.5 min; el FT solo se despega entrenando hasta 17.5 min. El feature extractor es el peor punto: más caro que la CNN (25× en inferencia, 1.5× en entrenamiento) y 4 puntos peor.
- **¿Por qué sirven features de 224×224 en imágenes de 32×32?** Las capas bajas aprenden detectores genéricos (bordes, colores, texturas, partes) y CIFAR-10 comparte casi todas sus categorías con ImageNet [7]. Pero ampliar la imagen la vuelve borrosa: sin alta frecuencia hay un domain shift que limita al backbone congelado a 0.877. Además, el mapa final es solo de 3×3. El costo crece con el área: 3.96 GMACs a 112 px frente a ~15.5 a 224 px.
- **Fine-tuning.** Más bloques descongelados mejoraron de forma monótona el F1 de validación: b5 0.901 → b4–5 0.926 → b3–5 0.934, a costa de 104 → 154 → 210 s/epoch y 1.75 → 2.32 GB. Bajar el lr del backbone de 1e-3 a 1e-4 redujo el F1 de 0.926 a 0.902: el backbone casi no se adapta en 5 epochs. No hubo catastrophic forgetting; solo overfitting leve (train 0.15 vs. val 0.19 en FT-4).
- **Menos datos.** Con el 10 %, la CNN desde cero fue la más afectada: F1 0.918 → 0.787 (–13.1 puntos). El FE cayó –5.7 (0.820) y el FT –5.1 (0.883), y ambos preentrenados superan a la CNN. Coincide con lo visto en clase: con pocos datos, feature extraction (dominio parecido, muy pocos datos) o fine-tuning de los bloques superiores; desde cero solo cuando hay muchos datos o el dominio es muy distinto.
- **Matrices de confusión.** Los tres modelos comparten el error dominante gato↔perro (15.4 % CNN, 20.9 % FE, 14.5 % FT, ambos sentidos). Le siguen automóvil↔camión y avión↔barco en la CNN y el FT, y venado↔caballo y pájaro↔venado en el FE. El FE reparte más errores entre animales; el FT los corrige salvo los pares semánticamente cercanos. Son los mismos errores, en distinta magnitud.
- **Despliegue.** (a) En un servidor con GPU, VGG-16 con fine-tuning: el mejor F1, 4.4 ms por imagen (1.6 ms en batch), y sus ~540 MB de pesos no son problema. (b) En un móvil, la CNN desde cero (2.2 M parámetros ≈ 9 MB, 0.15 GMACs), porque VGG-16 (≈537 MB) es inviable. Como preentrenada alternativa usaría MobileNetV3 (2.5–5.4 M parámetros, convoluciones depthwise separable) [8] o EfficientNet-B0, con fine-tuning y cuantización int8.

6. Conclusiones

- El **fine-tuning** de VGG-16 fue el mejor modelo (F1 test 0.934). Lo decisivo fue descongelar más bloques con un lr de backbone 10× menor que el de la cabeza.
- Con todos los datos, una **CNN pequeña bien regularizada** queda a 1.6 puntos con ~4 % del costo de inferencia: es la mejor opción costo-beneficio.
- El **feature extractor** congelado fue la peor opción a 112 px por el domain shift del redimensionamiento, aunque es el más estable con pocos datos.
- Con **pocos datos** el transfer learning gana claramente: la CNN desde cero pierde 13 puntos y los modelos preentrenados solo ~5.

Referencias

- [1] PyTorch. torchvision.models — VGG16 y VGG16_Weights. <https://pytorch.org/vision/stable/models/vgg.html>
- [2] K. Simonyan y A. Zisserman (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. ICLR.
- [3] PyTorch. Autograd mechanics; nn.Module.train/eval; torch.no_grad. <https://pytorch.org/docs/stable/notes/autograd.html>
- [4] PyTorch. torch.optim — per-parameter options. <https://pytorch.org/docs/stable/optim.html>
- [5] torchinfo (T. Yep) y fvcore (Meta AI Research), documentación oficial.
- [6] PyTorch. CUDA semantics — memory management; torch.mps. <https://pytorch.org/docs/stable/notes/cuda.html>
- [7] J. Yosinski et al. (2014). How transferable are features in deep neural networks? NeurIPS.
- [8] A. Howard et al. (2019). Searching for MobileNetV3. ICCV.